

Week 1: Data Cleaning and Feature Engineering Report

- Intern's Name : ALLAM SRIDHAR
- Date of Submission : 21-10-2024

The purpose of this report is to document the data preprocessing tasks completed during the week to prepare the dataset for further analysis. Data preprocessing is a critical step in the data analysis pipeline, as it enhances data quality, consistency, and readiness for Analytical Modelling. By performing various cleaning and transformation tasks, we aim to ensure that the dataset is suitable for generating insights and making data-driven decisions.

Data Overview:

- Number of rows : 8558
- Number of Columns : 33

Column Names :

Profile Id, Learner SignUp DateTime, Opportunity Id, Opportunity Name, Opportunity Category, Opportunity End Date, First Name, Last Name, Date of Birth, Gender, Mobile Number, Address Line 1, Address Line 2, City, State, Country, Zip Code, Institution Name, Graduation Date, Current Student Status, Current/Intended Major, Entry created at, Status Description, Status Code, Apply Date, Opportunity Start Date, Reward Awarded Date, Completion Date, Reward Amount, Badge Id, Badge Name, Skill Points Earned, Skills Earned

Column Analysis :

Profile Id :

- Data type: Object
- Unique Values: 3964
- Missing Values: 0

Learner SignUp DateTime:

- Data type: object
- Unique Values: 3916
- Missing Values: 0

Opportunity Id :

- Data type: object
- Unique Values: 23
- Missing Values: 0

Opportunity Name :

- Data type: object
- Unique Values: 22
- Missing Values: 0

Opportunity Category :

- Data type: object
- Unique Values: 5
- Missing Values: 0

Opportunity End Date :

- Data type: object
- Unique Values: 17
- Missing Values: 0

First Name :

- Data type: object
- Unique Values: 3026
- Missing Values: 0

Last Name :

- Data type: object
- Unique Values: 3092
- Missing Values: 0

Date of Birth :

- Data type: object
- Unique Values: 2620
- Missing Values: 0

Gender :

- Data type: object
- Unique Values: 4
- Missing Values: 0

Mobile Number :

- Data type:
- Unique Values: 3834
- Missing Values: 0

Address Line 1 :

- Data type:
- Unique Values: 3742
- Missing Values: 4

Address Line 2 :

- Data type: object
- Unique Values: 1552
- Missing Values: 5005

City :

- Data type:
- Unique Values: 1341
- Missing Values: 1

State :

- Data type: object
- Unique Values: 3
- Missing Values: 645

Country :

- Data type: object
- Unique Values: 71
- Missing Values: 0

Zip Code :

- Data type: object
- Unique Values: 1907
- Missing Values: 3

Institution Name :

- Data type: object
- Unique Values: 2089
- Missing Values: 5

Graduation Date :

- Data type: object
- Unique Values: 243
- Missing Values: 0

Current Student Status :

- Data type: object
- Unique Values: 4
- Missing Values: 0

Current/Intended Major :

- Data type: object
- Unique Values: 406
- Missing Values: 5

Entry created at :

- Data type: object
- Unique Values: 94
- Missing Values: 0

Status Description :

- Data type: object
- Unique Values: 8
- Missing Values: 0

Status Code :

- Data type: int64
- Unique Values: 8
- Missing Values: 0

Apply Date :

- Data type: object
- Unique Values: 8548
- Missing Values: 0

Opportunity Start Date :

- Data type: object
- Unique Values: 18
- Missing Values: 3794

Reward Awarded Date :

- Data type: object
- Unique Values: 8
- Missing Values: 8529

Completion Date :

- Data type: object
- Unique Values: 8
- Missing Values: 8529

Reward Amount :

- Data type: float64
- Unique Values: 2
- Missing Values: 8529

Badge Id :

- Data type: object
- Unique Values: 4
- Missing Values: 8529

Badge Name :

- Data type: object
- Unique Values: 4
- Missing Values: 8529

Skill Points Earned :

- Data type: float64
- Unique Values: 4
- Missing Values: 8529

Skills Earned :

- Data type: object
- Unique Values: 4
- Missing Values: 8529

Data Cleaning Process

Cleaning Steps:

Handling missing values

- Here I don't think Address Line 1, Address line 2 will be of much use, so I think of replacing them with Null/Not mentioned.
- There is 1 missing in city, 3 in state, 3 in zip codes, first I'll try to find their values through adjacent city, state, city/ address columns if found then okay else I'll remove them directly, since they are very few in numbers.
- There are 5 missing in Institution name and Current/Intended Major/Institution Name, most probably I'll try to group by city/country and then take most frequent in that country and replace with them.
- There are 3794 null values in the Opportunity start date, here since there is Opportunity End Date, I'll try to find if there is any relation between those columns and replace the null values with those relationship values (i.e. days difference).
- There are 8529 missing values in Reward Awarded Date, Completion Date, Reward Amount, Badge Id, Badge Name, Skills Points Earned, Skills Earned, actually these values are present only if the student has completed his course/opportunity, so since these values are missing indicates that, the remaining students have not completed the course/opportunity, so I like to replace with 0 for numerical cols and Null for categorical columns.
- Here from zip code: 12655 I know that it belongs to Cairo.
- And since Ghana is a country which is not divided into states but regions and here no zip codes (but sometimes they use 233 or 00223- which is their international calling code", but capital is ACCRA city, so we replace null values with Accra and zip codes with 233, in order to not to drop it.
- So in Ghana, replace state with ACCRA city, zip codes with 233.
- In Egypt, the zip codes 12655 belongs to Cairo State.
- In rest of zip codes since we don't know their values replacing them with 000 or dropping them off.
- Now for Opportunity Start Column we do have 3794 missing values, about that, since we already have Opportunity End Date column, I am trying to find the relation between the Opportunity Start column and Opportunity End column (i.e. Days in between them) and try to impute values based on this relation, I can achieve this with help of below code:

```

# Convert date columns to datetime
date_columns = ['Opportunity Start Date', 'Opportunity End Date']
for col in date_columns:
    df[col] = pd.to_datetime(df[col], errors='coerce')

# Handle Opportunity Start Date
# Calculate the median duration between start and end dates for non-null pairs
valid_duration = df['Opportunity End Date'] - df['Opportunity Start Date']
median_duration = valid_duration.dropna().median()

# Fill missing start dates by subtracting the median duration from the end date
df.loc[df['Opportunity Start Date'].isnull(), 'Opportunity Start Date'] = df['Opportunity End Date'] - median_duration

```

Finding and removing duplicates:

- For finding duplicates we use `df.duplicated().sum()`
- To remove them we use `df = df.drop_duplicates()`
- **Note:** But there are no duplicates present in the dataset.

Standardizing Formats:

- Opportunity End Date, Date of Birth, Graduation Date, Apply Date, Opportunity Start Date, Reward Awarded Date, Completion Date, Learner SignUp DateTime columns are in Object data type, first we need to convert it to the date-time format and extract only date from that and store it back in the same column.

For **Opportunity End Date** column, we can achieve it with help of below code:

```

# Extract only the date part
df['Opportunity End Date'] = df['Opportunity End Date'].dt.date

```

For **Date of Birth** column, the problem is that the dates are stored in different formats in that column like some dates are stored in 01-12-2001 format and some are stored in 21/10/2002 format, so I am converting them both into date format with help of below code:

```

# Define a function to convert the date formats
def convert_date_format(date):
    for fmt in ("%m/%d/%Y", "%d-%m-%Y"): # Add more formats if needed
        try:
            return pd.to_datetime(date, format=fmt)
        except ValueError:
            continue
    return pd.NaT # Return NaT if no format matches

```

```
# Apply the function to the 'Date of Birth' column
df['Date of Birth'] = df['Date of Birth'].apply(convert_date_format)
```

```
# Extract only the date part
df['Date of Birth'] = df['Date of Birth'].dt.date
```

For **Graduation Date** column, here in this column there is one value wrongly entered as 01/0205 instead of 2005, so first replace it with correct value and then convert it to date format, we can achieve this with below code:

```
# Replace invalid date(s)
df['Graduation Date'] = df['Graduation Date'].replace('01/0205', '01/2025')
# Convert to datetime format
df['Graduation Date'] = pd.to_datetime(df['Graduation Date'], format='%m/%Y', errors='coerce')
```

For **Apply Dates** column, here the values are in different formats again like 06/14/2023 12:36:09 and 05-01-2023 06:08:21, so here we Split on space and take the first part, then Replace hyphens with slashes for consistency, then convert it to date-time and extract the date from it, we can achieve it with below code:

```
# Step 1: Clean and extract valid date formats
cleaned_dates = []
for date_str in df['Apply Date']:
    # Split on space and take the first part
    date_part = date_str.split()[0]

    # Replace hyphens with slashes for consistency
    date_part = date_part.replace('-', '/')

    # Convert to datetime
    try:
        date = pd.to_datetime(date_part, errors='coerce')
        cleaned_dates.append(date.date() if pd.notna(date) else None)
    except Exception:
        cleaned_dates.append(None)

# Assign the cleaned dates back to the 'Apply Date' column
df['Apply Date'] = cleaned_dates
```

For **Opportunity Start Date**, **Reward Awarded Date**, **Completion Date** columns we can convert them with below code directly

```
df['Opportunity Start Date'] = pd.to_datetime(df['Opportunity Start Date']).dt.date
df['Reward Awarded Date'] = pd.to_datetime(df['Reward Awarded Date']).dt.date
df['Completion Date'] = pd.to_datetime(df['Completion Date']).dt.date
```

For **Learner SignUp DateTime** column again here the values are stored in different formats similar to Apply Date column, so we can solve this problem similar to that column:

```
# Step 1: Clean and extract valid date formats
cleaned_signup_dates = []
for date_str in df['Learner SignUp DateTime']:
    # Split on space and take the first part (date)
    date_part = date_str.split()[0]

    # Replace hyphens with slashes for consistency
    date_part = date_part.replace('-', '/')

    # Convert to datetime and extract the date
    try:
        date = pd.to_datetime(date_part, errors='coerce')
        cleaned_signup_dates.append(date.date() if pd.notna(date) else None)
    except Exception:
        cleaned_signup_dates.append(None)

# Assign the cleaned dates back to the 'Learner SignUp DateTime' column
df['Learner SignUp DateTime'] = cleaned_signup_dates
```

Also My numerical column like Reward Amount and Skill Points Earned columns are in float64 type, I am converting them into integer format using below code:

```
df['Reward Amount'] = df['Reward Amount'].fillna(0).astype(int)
df['Skill Points Earned'] = df['Skill Points Earned'].fillna(0).astype(int)
```

Data Validation

- In **countries** column, for the same country name they have been written differently like: Iran Islamic Republic of Persian Gulf and Iran, Islamic Republic of Persian Gulf. so I am replace both of them with Iran, so I did correct it using the below code:

```
# Define a dictionary with variations to standardize
country_replacements = {
    'Iran Islamic Republic of Persian Gulf': 'Iran',
    'Iran, Islamic Republic of Persian Gulf': 'Iran'}
# Replace the variations in the 'Countries' column
df['Country'] = df['Country'].replace(country_replacements)
# Confirm the replacements
print(df['Country'].value_counts())
```

In **states column**, same state name have been entered in different format like: Abia, Abia state; Adamawa, Adawama state; etc... So I am replacing those with one of the given types among them with help of below code:

```
# Define the mapping for states
```



```

state_mapping = {
    'Abia': ['Abia', 'Abia State'],
    'Abuja': ['Abuja Capital Territory', 'Abuja', 'Abuja FCT', 'Federal Capital Territory', 'FCT', 'Federal
capital territor'],
    'Adamawa': ['Adamawa', 'Adamawa State Nigeria'],
    'Akwa Ibom': ['Akwa Ibom', 'Akwa Ibom state'],
    'Andhra Pradesh': ['Andhra Pradesh', 'Andhra pradesh', 'Andhra p', 'Andhra Pradesh state',
'Andhrapradesh', 'Andra pradesh'],
    'Ashanti': ['Ashanti region', 'Ashanti', 'Ashanti Re'],
    'Borno': ['Borno', 'Borno State'],
    'Cross River': ['Cross River', 'Cross River State'],
    'Damietta': ['Damietta', 'dameitta'],
    'Delhi': ['Delhi', 'Del', 'New dehi', 'New Delhi'],
    'Delta': ['Delta', 'Delta State'],
    'Dhaka': ['Dhaka City', 'Dhaka Division', 'dhaka', 'Dhaka North'],
    'Edo': ['Edo', 'Edo State'],
    'Ekiti': ['EKITI', 'Ekiti State', 'Ekito'],
    'Enugu': ['Enugu', 'Enugu state'],
    'FCT': ['FCT'], # Only for Pakistan
    'Greater Accra': ['Greater Accra', 'Greater Accra Region', 'Greater AccraRegion', 'Greater
AccraRegion p', 'Accra'],
    'Gujarat': ['Gujarat', 'gujrat'],
    'Haryana': ['Haryana', 'Haryan', 'Haryans'],
    'Illinois': ['Illinois', 'Illiniois', 'illionis', 'ILLINOIS'],
    'Imo': ['Imo', 'Imo state'],
    'Islamabad': ['Islamic Republic of Pakistan', 'Islamic'],
    'Kampala': ['Kampala', 'Kampala city'],
    'Kano': ['Kano', 'Kano State'],
    'Karnataka': ['Karnataka', 'Karnatak', 'Karanataka', 'Karnataka state'],
    'Khyber Pakhtunkhwa': ['Khyber Pakuntunkhwa', 'Khyber Patun'],
    'Kigali': ['Kigali', 'Kigali city'],
    'Kogi': ['Kogi', 'Kogi state'],
    'Kwara': ['Kwara', 'Kwara state'],
    'Lagos': ['Lagos', 'Lagos state'],
    'Madhya Pradesh': ['MADHYA PRADESH', 'Madhya Pradesh', 'M P', 'MP'],
    'Madhesh Pradesh': ['Madhesh Pradesh', 'Madhesh Province'],
    'Maharashtra': ['Maharashtra', 'Maharashtr', 'Maharashtra', 'Maharasthra', 'maharastra',
'maharshtra', 'Mharashthra'],
    'Missouri': ['Missouri', 'MISSOURI', 'Missouri state', 'Missiori', 'Missori', 'missori', 'Missour',
'missuori', 'Missoure', 'missou', 'MissouriMissouru', 'Saint Louis', 'Saint Louis City', 'St Louis'],
    'Nassarawa': ['Nassarawa', 'Nasarawa', 'Nassarawa State'],
    'Niger': ['Niger', 'Niger state'],
    'Northern Region': ['Northern region', 'Northern R', 'Northern'],
    'Ogun': ['Ogun', 'Ogun Stat', 'Ogun State'],
    'Ondo': ['Ondo', 'Ondo state'],
    'Osun': ['Osun', 'Osun State'],
    'Oyo': ['Oyo', 'Oyo N', 'Oyo st', 'Oyo State'],
    'Pakistan': ['Pakistan', 'Pakistan k', 'Pak', 'Pakista', 'pakistan'],

```

```

'Plateau': ['Plateau', 'Plateau state', 'Plateau States'],
'Punjab': ['Province punjab', 'Pakistan Punjab', 'Punjab', 'PUNJAB', 'punjab'], # Only for Pakistan
'Puducherry': ['Pondicherry', 'Puducherry'],
'Rivers': ['Rivers', 'River State', 'Rivers state', 'RiversState'],
'Sokoto': ['Sokoto', 'SOKOTO STATE'],
'Tamil Nadu': ['Tamil Nadu', 'Tamil Nadu state', 'Tamilnadu', 'TN'],
'Telangana': ['Telangana', 'Tel', 'TELANGAN', 'TELANGANA', 'telangan', 'Telangana', 'Telangana state',
'Telegana', 'Telengana', 'telengana', 'telanngana', 'telangana', 'Telanagana'],
'Uttar Pradesh': ['Uttar Pradesh', 'UP', 'UTTAR PRADESH', 'Uttar Prades', 'up', 'uttar pradesh', 'up'],
'Uttarakhand': ['Uttarakhand', 'Uttrakhand'],
'West Bengal': ['West Bengal', 'WB', 'WestBengal']
}

```

Function to standardize state names based on the country

```

def standardize_state(row):
    for key, variations in state_mapping.items():
        if row['State'] in variations:
            if row['Country'] == 'Pakistan' and key in ['FCT', 'Punjab']:
                return key
            return key
    return row['State'] # Return original if no match found

```

Apply the function to standardize the 'State Name' column

```
df['State'] = df.apply(standardize_state, axis=1)
```

Check the updated DataFrame

```
print(df[['State', 'Country']])
```

For city column:

Especially for Hyderabad city in some places country name or city name are wrongly typed, so correcting using below code:

```
df.loc[(df['City'] == 'Hyderabad') & (df['Country'] != 'Pakistan'), ['State', 'Country']] = ['Telangana', 'India']
```

In city column, same city name have been entered in different format like: ADO, Ado Ekiti; Akure, Akure City; etc... So I am replacing those with one of the given types among them, similar to the countries column, we can achieve it with below code:

Define the mapping for cities

```

city_mapping = {
    'Addis Abeba': ['Addis Abeba', 'addis ababa', 'Addsis Ababa'],
    'ADO': ['ADO', 'Ado Ekiti'],
    'Agbara': ['Agbara', 'Agbara edu'],
    'Ahmedabad': ['Ahmedabad', 'AHMEDABAD', 'Ahmedabad'],

```

'Akure': ['Akure', 'Akure City'],
'AMBERNATH': ['AMBERNATH', 'Ambarnath'],
'Anantapur': ['Anantapur', 'Ananthapur'],
'Ayobo': ['Ayobo', 'Ayoboo'],
'Ballwin': ['Ballwin', 'Ballwin Manchester'],
'Bangalore': ['Bangalore', 'Bangalore South', 'Bangalore Urban', 'Banglore'],
'Bengaluru': ['Bengaluru', 'Bengaluru city', 'Bengaluru Urban'],
'Benin': ['Benin', 'Benin City'],
'Bidar': ['Bidar', 'Bida'],
'Bokaro': ['Bokaro', 'Bokaro Steel City'],
'Brooklyn': ['Brooklyn'],
'Can Tho': ['Can Tho', 'Cantho'],
'Chennai': ['Chennai', 'Chennai Region', 'Chennnai'],
'Chicago': ['Chicago', 'Chicag'],
'Delhi': ['Delhi', 'Del', 'New dehi', 'New delhi'],
'Edo': ['Edo', 'Ede'], # Ede should be checked if it's correct.
'Eluru': ['Eluru', 'Eluru city'],
'FCT': ['FCT', 'Federal Capital Territory'],
'Greater Noida': ['Greater Noida', 'Greater Noida west'],
'Gurugram': ['Gurugram', 'Gurgram'],
'Hanamakonda': ['Hanamakonda', 'Hanumakonda', 'Hanamakonda'],
'Ho Chi Minh City': ['Ho Chi Minh City', 'Ho'],
'Hyderabad': ['Hyderabad', 'hyderabad', 'Hyd', 'HYDERABAD', 'hyderabad', 'Hydrabad', 'Hyderbad',
'Hydeabad', 'Hyderaba', 'Hyderabad', 'Hyderavad'],
'Ibadan': ['Ibadan', 'Iba', 'Ibadan city', 'Ibadan N'],
'Ifako': ['Ifako', 'Ifako ijaiye'],
'Ikorodu': ['Ikorodu', 'Ikorodududu'],
'Isolo': ['Isolo', 'Isolo road'],
'Jayashankar bhupalapally': ['Jayashankar bhupalapally', 'Jayashankar bhupalapally'],
'Jimeta': ['Jimeta', 'Jimeta Yola'],
'Jos city': ['Jos city', 'Jos', 'Jos south'],

'Kalyan': ['Kalyan', 'Kalyan west'],
'KANDUKUR': ['KANDUKUR', 'Kandukuru'],
'Kanpur': ['Kanpur', 'Kanpur Dehat District', 'Kanpur Nagar'],
'Khairpur': ['Khairpur', 'Khairpur Mirs'],
'Kigali': ['Kigali', 'Kigali city'],
'Lagos': ['Lagos', 'Lagos City', 'Lagos s', 'Lagos state'],
'Lahore': ['Lahore', 'Lahore Pak'],
'Mandanapalli': ['Mandanapalli', 'Mandanapalle'],
'Mahbubnagar': ['Mahbubnagar', 'mahabubnagar'],
'Mandi': ['Mandi', 'Mandi bahauddin'],
'Mangalore': ['Mangalore', 'Mangaluru'],
'MARYLAND HEIGHTS': ['MARYLAND HEIGHTS', 'Maryland heights stlouis'],
'Minna': ['Minna', 'Minne'],
'missouri': ['missouri'],
'Mumbai': ['Mumbai', 'Mumbai City', 'Navi mumbai'],
'Mysore': ['Mysore', 'Mysuru'],
'Nairobi': ['Nairobi', 'Nairobi Municipality'],
'Muzaffarnagar': ['Muzaffarnagar', 'MUZAFFARPUR', 'Muzaffarpur'],
'NANDYAL': ['NANDYAL', 'NANDYAL KURNOOL'],
'Narasaraopeta': ['Narasaraopeta', 'Narasaraopet'],
'New Dameitta': ['New Dameitta', 'new dameitta'],
'New Okooba': ['New Okooba', 'New oko oba'],
'Nizamabad': ['Nizamabad', 'Nizamabad District'],
'Obuase': ['Obuase', 'Obuasi'],
'Ondo': ['Ondo', 'Ondo city'],
'Owerri': ['Owerri', 'Owerri municipal'],
'Pakistan': ['Pakistan', 'Pakistam'],
'Patan': ['Patan'],
'Port Harcourt': ['Port Harcourt', 'Port Harcourt city', 'Portharcourt'],
'Rajampeta': ['Rajampeta', 'Rajampet'],
'Roorke': ['Roorke', 'roorkee'],

```

'Saint Louis': ['Saint Louis', 'saint lo', 'Saint Louis c', 'Saint Louis city', 'Saint Louis l',
               'Saint Louis university', 'Saint Louise', 'saint lousis', 'Saintloui', 'Saintlouis', 'Saintt Louis',
               'SAIT LOUIS', 'sanit lousis', 'St', 'St Loius', 'St lousis', 'St Louis City', 'stl', 'stLouis'
               'SAINT LOUIS', 'St Louis'],
'St charles': ['St charles', 'StCharles', 'St Charles'],
'Sathupally': ['Sathupally', 'Sathupalli'],
'Sattenapalli': ['Sattenapalli', 'SATTENAPALLE', 'Sattenpalli'],
'Sharjah': ['Sharjah', 'Sharjha'],
'Sheikhupura': ['Sheikhupura', 'Sheikupura'],
'Sonipat': ['Sonipat', 'sonepat'],
'Tarkwa': ['Tarkwa', 'Tarkwa Tamso'],
'Thane': ['Thane', 'THANE WEST'],
'Tirupathi': ['Tirupathi', 'Tirupati'],
'VISHAKHAPATANAM': ['VISHAKHAPATANAM', 'Visakhapatnam', 'Vishakhapatnam'],
'Vissanapet': ['Vissanapet', 'vissannapeta'],
'warangal': ['warangal', 'waranagal', 'Warangal urban', 'Warangal rural'],
'Zaria': ['Zaria', 'Zaria City']
}

```

```

# Function to standardize city names based on the country

```

```

def standardize_city(row):

```

```

    for key, variations in city_mapping.items():

```

```

        if row['City'] in variations:

```

```

            if row['City'] == 'Hyderabad' and row['Country'] != 'Pakistan':

```

```

                # Change the state name and country if city is Hyderabad

```

```

                row['State'] = 'Telangana'

```

```

                row['Country'] = 'India'

```

```

            return key # Return the standardized city name

```

```

        return row['City'] # Return original if no match found

```

```

# Apply the function to standardize the 'City Name' column

```

```

df['City'] = df.apply(standardize_city, axis=1)

```

```
# Check the updated DataFrame
```

```
print(df[['City', 'Country']])
```

For **zip codes** column if Country is Ghana, then if the values are numbers then replace them with 233, else if they are alphabets/alphanumeric/characters present then extract first 3/2 alphabets only, if there are 3 alphabets in the beginning extract those 3 else extract first 2 character and place them in that column.

if it is alpha numeric like numeric followed by space or alphabet then remove those space and alphabets and only take numbers, we can achieve it with below code:

```
# Function to process the 'Zipcode' column based on country
```

```
def process_zipcode(row):
```

```
    if row['Country'].strip().lower() == 'ghana':
```

```
        zipcode = str(row['Zip Code']).strip()
```

```
        # Check if the value is numeric
```

```
        if zipcode.isnumeric():
```

```
            return '233'
```

```
        else:
```

```
            # Extract first 3 characters if all are alphabets
```

```
            if zipcode[:3].isalpha():
```

```
                return zipcode[:3].upper()
```

```
            # Extract first 2 characters otherwise
```

```
            else:
```

```
                return zipcode[:2].upper()
```

```
    return row['Zip Code'] # Keep the original value if not Ghana
```

```
# Apply the function to the 'Zipcode' column
```

```
df['Zip Code'] = df.apply(process_zipcode, axis=1)
```

- Now if in **zip column** if the values are in the format 60614-4904 then only consider 60614 from it.
- if they are in this format 485 garki then only take 485 from it.
- if they are in the format Saifali1163 or WA189932 or Po box 10410 or MO 63108 then only take 1163 or 189932 or 10410 10410 from it.
- if there are any spaces in between like this 625 020 or 600 127 or LS7 2RQ, then remove the space and consider it as 625020 or 600127 or LS72RQ.
- else let them be as they are, we can achieve it with below codes:

```
import re
```

```
# Function to process the 'Zipcode' column
```

```
def process_zipcode(row):
```

```
    zipcode = str(row['Zip Code']).strip()
```

```
    # Case 1: Format 60614-4904 -> Take first part before '-'
```

```
    if '-' in zipcode:
```

```
        return zipcode.split('-')[0]
```

```

# Case 2: Format "485 garki" -> Extract numeric part only
elif re.match(r'^\d+\s*\D+', zipcode):
    return re.findall(r'^\d+', zipcode)[0]

# Case 3: Format like "Saifali1163", "WA189932", "Po box 10410", "MO 63108"
elif re.search(r'\d+', zipcode):
    return re.findall(r'\d+', zipcode)[-1]

# Case 4: If spaces between numeric values, e.g., 625 020 -> Remove spaces
elif re.match(r'^\d+\s+\d+$', zipcode):
    return zipcode.replace(' ', '')

# Default: Leave as-is
return zipcode
# Apply the function to the 'Zipcode' column
df['Zip Code'] = df.apply(process_zipcode, axis=1)

```

In **Institution Name** column also we can see the same same problem as in State,city column, so I am using the same technique to solve the problem, but I am not adding the codes here since its too large and the replacements are in large number.

But before the transformation total unique columns were 2089, but after transformation they are 1761.

Feature Engineering

There are some existing columns which needed to be merged like **First Name, Last Name**

And also Address Line 1, Address Line 2, so I am merging them, with help of below codes:

```

df['Full_Address'] = df['Address Line 1'].fillna("") + ' ' + df['Address Line 2'].fillna("")
df['Full_Address'] = df['Full_Address'].str.strip() # Remove extra spaces if any
df['Full_Name'] = df['First Name'].fillna("") + ' ' + df['Last Name'].fillna("")
df['Full_Name'] = df['Full_Name'].str.strip() # Remove extra spaces if any

```

Creating New column called **Age**, which I am calculating with help of Date of Birth column and datetime.now() function, with help of below code:

```

# Ensure 'Date of Birth' is in datetime format
df['Date of Birth'] = pd.to_datetime(df['Date of Birth'], errors='coerce')
# Calculate the age in years
df['Age'] = (datetime.now() - df['Date of Birth']).dt.days // 365

```

Also creating **Opportunity Duration (days)** column to know how many days the opportunity is available for user once signed up.

1. Calculate Opportunity Duration (in Days)

```
df['Opportunity Duration (days)'] = (df['Opportunity End Date'] - df['Opportunity Start Date']).dt.days
```

Also creating **Time to Apply(days)** column to know the days difference between when applied on and when his entry was created.

2. Time Taken to Apply (in Days)

```
df['Time to Apply (days)'] = (df['Entry created at'] - df['Apply Date']).dt.days
```

In Address column some people have entered only numbers like phone numbers but they are not actual addresses so I am replacing such values with No address, with help of below code:

Replace numeric-only entries with 'No address' or 'Wrong address'

```
df['Full_Address'] = df['Full_Address'].apply(lambda x: 'No address' if x.isdigit() else x)
```

I am dropping some columns like **First Name, Last Name, Address Line 1, Address Line 2, Status Code** etc since we have that exact data in other columns. We can achieve it using

```
df = df.drop(['First Name', 'Last Name', 'Address Line 1', 'Address Line 2', 'Status Code'], axis=1)
```

Finally saved all the Pre-processed data to csv file using the below code:

Save the updated DataFrame to a CSV file

```
df.to_csv(r"C:\Users\vamsh\Downloads\Cleaned_Preprocessed_Dataset_Week1.csv", index=False)  
print("Dataset saved successfully")
```

Conclusion:

So in this Entire week we did all the preprocessing steps mentioned above till now and processed the data, now this data is ready to do the next steps like performing EDA, finding patterns and insights from the Data, also to do Visualizations on the data.